



Zoo 3.0

GPU-Native L1: Fastest Blockchain Ever Created

Antje Worring & Zach Kelling

Zoo Labs Foundation

December 15, 2025

Abstract

ZOO 3.0 is the GPU-native graduation of the ZOO stack. Every hot-path primitive — consensus signature aggregation, BLS12-381 verification, RINGTAIL threshold proofs, ML-DSA-65 identity proofs, EVM bytecode execution, AMM curve math, FHE ciphertext arithmetic, and Groth16/Halo2 ZKP verification — runs as deterministic GPU kernels in pure C++ via the QUASARGPU adapter (LP-132) under the GPU-Residency Invariant (LP-137). With this release, ZOO graduates from a LUX L2 application to a sovereign LUX L1 with its own validator set, QUASAR-certified consensus, and independent economic security, while continuing to settle and bridge to the broader LUX chain topology (LP-134). This paper documents the 3.0 architecture: the L1 graduation, the GPU-native execution stack, the ZOO DEX (native AMM V2/V3, V4 as a precompile to the LUX DEX), the ZOO AI Desktop application hosting the ZEN model family for free local private inference, the `zoo-bot` pure-Go re-write of the HANZO OpenClaw bot, the ZOO Bridge multi-ecosystem connector, dual-path privacy via F-Chain FHE (LP-013) and Z-Chain ZKP (LP-063), and the NFT migration program. Spec freeze: 2025-12-15. Activation alongside LUX QUASAR 3.0: 2025-12-25.

Keywords: ZOO 3.0, GPU-native, L1 graduation, QUASAR 3.0, ZOO DEX, ZOO AI Desktop, `zoo-bot`, ZOO Bridge, FHE, ZKP, NFT migration

Contents

1	The 3.0 Thesis	5
1.1	What “GPU-native” actually means	5
1.2	Why this is a category step, not a tuning pass	5
1.3	What ZOO 3.0 is not	5
1.4	Why now	6
2	Graduating to L1	6
2.1	What changes	6
2.2	Why graduate	6
2.3	Constructive embedding in Lux topology	7
2.4	Validator economics	7
3	GPU-Native Across the Stack	7
3.1	Consensus (QUASAR 3.0, LP-020 + LP-135)	7
3.2	EVM execution (LP-009)	8
3.3	AI mining	8
3.4	DEX matching	8
3.5	F-Chain FHE (LP-013)	8
3.6	Z-Chain ZKP (LP-063)	8
3.7	The discipline	9
4	ZOO DEX	9
4.1	V2 — constant product	9
4.2	V3 — concentrated liquidity	9
4.3	V4 — LUX DEX precompile	9
4.4	Optional local Zoo DEX precompile	10
4.5	Per-LLM token markets	10
4.6	NFT fractional pools	10
4.7	Versioning policy	10

5	Zoo AI Desktop and Local Private AI	10
5.1	Form factor	10
5.2	Free local inference	10
5.3	Zoo Cloud upgrade path	11
5.4	Privacy posture	11
5.5	Mining UX	11
5.6	Why this matters for Zoo 3.0	11
6	zoo-bot	11
6.1	Why a re-write	11
6.2	Goals	12
6.3	Architecture	12
6.4	Use cases	12
6.5	Security model	12
6.6	Relationship to OpenClaw	13
7	Zoo Bridge	13
7.1	Architecture	13
7.2	Supported chains	13
7.3	Conservation metadata preservation	13
7.4	Yield-bearing bridge bookkeeping	14
7.5	Security model	14
7.6	Migration of 2021 ZOO BSC deployment	14
8	Privacy	14
8.1	F-Chain FHE (LP-013)	14
8.2	Z-Chain ZKP (LP-063)	15
8.3	Choosing between them	15
8.4	GPU-native privacy	16
9	NFT Strategy	16
9.1	Why a collection should migrate	16
9.2	Migration mechanics	16
9.3	What ZOO offers a migrating collection	17
9.4	Categories	17
9.5	Note on naming	17
10	LUX Wallet and Desktop	17
10.1	Account / portfolio view	17
10.2	Send / receive	18
10.3	DEX swap	18
10.4	Staking	18
10.5	Bridge	18
10.6	AI inference	18
10.7	Privacy mode	19
10.8	Mobile and desktop parity	19
10.9	Screen inventory summary	19

11 Activation	19
11.1 Phase 0 — spec freeze (2025-12-15)	19
11.2 Phase 1 — testnet bring-up (2025-12-15 through 2025-12-22)	19
11.3 Phase 2 — canary (2025-12-22 through 2025-12-24)	20
11.4 Phase 3 — activation (2025-12-25)	20
11.5 Phase 4 — post-activation roll-forward	20
11.6 Cross-paper relationships	20
11.7 One-line summary	20

1 The 3.0 Thesis

ZOO 3.0 is 100% GPU-powered. The fastest blockchain plus consensus plus EVM ever shipped. Pure C++ and GPU kernels for all hot-path cryptography. No interpreter glue on the critical path. No CPU fallbacks gating throughput. No hybrid scheduler that pretends.

1.1 What “GPU-native” actually means

In the ZOO 3.0 stack, every primitive whose latency matters at validator scale lives on the GPU device for the duration of a block. Concretely:

- Triple-consensus signature verification (BLS12-381 aggregate + RINGTAIL threshold + ML-DSA-65 identity) runs as fused GPU kernels per the QUASARGPU adapter (LP-132).
- The EVM bytecode interpreter executes as a fiber VM with 256-bit U256 arithmetic implemented as device-resident SIMD (LP-009).
- Cold-state SLOAD suspends the fiber rather than blocking the warp, so a single warp finishes a block of mixed-cost transactions without serial stalls (LP-009).
- The GPU-Residency Invariant (LP-137) forbids spurious host round-trips: state, witnesses, and proofs stay on-device until the QUASARCert is emitted.

1.2 Why this is a category step, not a tuning pass

Block production is a pipeline of $O(n)$ verification + $O(n \log n)$ ordering + $O(n)$ execution + $O(1)$ certificate generation, where n is the transaction count. Optimizing any single stage on CPU eventually hits the memory wall or the per-core ceiling. ZOO 3.0 takes the entire pipeline to a SIMT machine where verification, execution, and proof emission share device memory. The result is not a constant-factor improvement; it is a different point on the latency-versus-throughput curve, anchored by three specs:

1. **LP-132 (QUASARGPU adapter)** — deterministic kernel scheduling that makes GPU execution reproducible across validators with the same hardware class.
2. **LP-009 (GPU-native EVM)** — fiber VM for bytecode, U256 arithmetic, suspend-on-cold-state semantics.
3. **LP-137 (GPU-Residency Invariant)** — the negative constraint: no host bounce on the hot path.

1.3 What Zoo 3.0 is not

ZOO 3.0 is not a marketing rename. It is not an upgrade only to consensus or only to the EVM. It is the substrate change. Every component documented in this paper — DEX matching, FHE compute, ZKP verification, AI inference attestation, bridge message verification, and wallet-side transaction signing — runs on the new substrate.

1.4 Why now

The Zen model family ships frontier-tier weights at every scale from **zen4-nano** (0.8B) through **zen4-ultra-max** (1T+). Modern data-center GPUs ship with confidential compute and FP4/FP8 tensor cores. QUASAR 3.0 ships with triple-consensus support, certificate parallelism. Together, these pieces close the loop: a chain that mints, trades, attests, and pays for AI compute on the same device class that runs the AI compute. ZOO 3.0 is the first chain to take that loop seriously.

2 Graduating to L1

Through 2.x, ZOO ran as a LUX L2 application: an EVM-compatible chain (Chain ID 200200) settling state commitments to LUX’s C-Chain. With 3.0 ZOO graduates to a sovereign L1 with its own validator set, its own QUASARCert finality, and its own economic security parameters, while remaining inside the LUX chain topology.

2.1 What changes

Concern	Zoo 2.x (L2)	Zoo 3.0 (L1)
Validator set	inherited from Lux	sovereign Zoo validators
Finality	via Lux C-Chain	QUASARCert local finality
Economic security	rented from Lux	\$ZOO + \$AI staking
Slashing	N/A	enforced by Q-Chain ceremony
Block production	sequencer	full BFT replication
Bridge to Lux	state commitments	B-Chain (LP-134)
Settlement guarantees	rollup proof	sovereign certificate
Native gas asset	ZOO (bridged)	ZOO (native)

2.2 Why graduate

Determinism for AI workloads. Per-LLM chain execution [12] requires deterministic scheduling of training-data attestations, attribution graph updates, and inference receipt rollups. Running these as L2 transactions inside a generic L1 sequencer was a solvable problem in 2.x; running them as native operations on a sovereign chain whose mempool, consensus, and finality are tuned for AI is materially better.

Native confidential compute. ZOO 3.0 exposes F-Chain-backed FHE (LP-013) and Z-Chain-backed ZKP (LP-063) as first-class precompiles. As an L2, every confidential operation paid the cross-domain message tax. As an L1 with chain-local precompiles, the same operations are a single transaction.

Sovereign tokenomics. Zoo’s PoAI ledger and AI mining rewards need a chain where the validator subsidy schedule, fee burn policy, and \$ZOO/\$AI dual-asset gas semantics are governance variables, not L2 constants inherited from upstream. L1 graduation puts these levers where the Zoo DAO can move them.

2.3 Constructive embedding in Lux topology

L1 graduation is not departure. The QUASAR-Native App Stack (LP-133) defines exactly how a sovereign appchain settles into the broader LUX topology (LP-134):

- **Q-Chain** runs the RINGTAIL threshold ceremonies that validate ZOO’s validator key material.
- **A-Chain** (AIVM) anchors TEE quotes for AI mining receipts; ZOO validators submit attestations there.
- **B-Chain** (BVM) carries ZOO↔LUX bridge messages and ZOO↔external-chain transfers via ZOO Bridge (§7).
- **F-Chain** (FVM) and **Z-Chain** (ZVM) provide confidential compute substrate that ZOO dispatches to via precompiles.

This is the appchain pattern: sovereign block production, shared cross-chain plumbing.

2.4 Validator economics

Validators stake \$ZOO and optionally \$AI. Block rewards are paid in the gas asset of the block. Quasar 3.0 triple-consensus (LP-020: BLS + Ringtail + ML-DSA) means every signed block carries both a classical fast path proof and a post-quantum proof; validators running compromised classical keys still cannot forge blocks because the RINGTAIL layer must independently authorize. Slashing is enforced via Q-Chain ceremonies for double-sign and via A-Chain attestation gaps for liveness failures.

3 GPU-Native Across the Stack

This section breaks down what runs on the GPU in ZOO 3.0 and why each component is on the device rather than the host. The design discipline is LP-137’s GPU-Residency Invariant: data on the hot path stays on the device until a certificate is emitted.

3.1 Consensus (QUASAR 3.0, LP-020 + LP-135)

QUASAR 3.0 ships triple-consensus:

- **BLS12-381** — 48-byte aggregate signature, classical fast path. Pairing check and miller-loop kernels run on GPU.
- **RINGTAIL (Ring-LWE)** — post-quantum threshold proof. GPU-resident polynomial multiplication (NTT) for signature share aggregation.
- **ML-DSA-65** (FIPS 204) — post-quantum identity proof (~3.3 KB). Hash-and-sample, polynomial arithmetic, and rejection sampling all on-device.

Each layer is independently toggleable via `TripleSignRound1`; the default 3.0 mode runs all three in parallel and emits a `QUASARCert` that carries all three proofs. Photon (committee selection), Wave (threshold voting), and Focus (confidence accumulation) execute as device-side logic per LP-105.

3.2 EVM execution (LP-009)

The ZOO 3.0 EVM is a fiber VM:

- Each transaction maps to a GPU fiber.
- U256 arithmetic uses 4×64 limb representation in registers with carry-chain primitives.
- Cold-state **SLOAD** suspends the fiber and yields the warp slot to a runnable peer; the storage-fetch coalescer batches outstanding loads into a single device-side trie traversal.
- Block-STM-style optimistic concurrency [12] (cf. LP-010) provides parallel execution with deterministic re-execution on conflict.

3.3 AI mining

The ZOO AI mining loop binds compute to receipts:

1. A miner runs Zen inference inside a TEE-attested container (NVIDIA H100 confidential compute, AMD SEV-SNP, Intel TDX).
2. The TEE quote plus an inference digest is submitted to the LUX A-Chain (AIVM, LP-134); the A-Chain anchors the receipt.
3. ZOO reads the A-Chain receipt and attributes the inference to the appropriate per-LLM chain ([12]), updating the contributor attribution graph.
4. Rewards are paid in \$AI to the miner and routed through the per-LLM chain’s distribution policy to upstream contributors (training-data, fine-tuners, evaluators).

The hot path of this loop — attestation verification, signature aggregation, attribution graph update — runs as GPU kernels.

3.4 DEX matching

ZOO’s DEX (see §4) implements AMM V2/V3 natively on the chain and exposes V4 as a precompile bridge to the LUX DEX [15]. AMM curve math, fee calculations, tick crossings, and order matching execute on the GPU. The constant-product invariant for V2 and the concentrated-liquidity tick math for V3 are simple enough that a single warp can step thousands of swaps per microsecond at validator hardware budgets.

3.5 F-Chain FHE (LP-013)

When a ZOO contract dispatches to the F-Chain FHE precompile, the ciphertext arithmetic (TFHE bootstrapping, key-switching, gadget products) runs on the F-Chain validator’s GPU. Confidential ERC-20 operations (LP-067) — transfer, balance check, allowance update — are device-side ciphertext circuits.

3.6 Z-Chain ZKP (LP-063)

Verification of Groth16, Halo2, and Plonky2 proofs runs as GPU kernels on the Z-Chain validator. ZOO dispatches verification requests via precompile; the Z-Chain returns a verification receipt that the calling ZOO contract treats as a boolean.

3.7 The discipline

The common pattern across all of the above: data lands on the GPU when a transaction enters the mempool, executes its lifecycle on the device (verify, run, prove, log), and only leaves the device when a certificate is committed to disk or sent over the wire. The CPU becomes the orchestration layer; the GPU is where the ledger lives during the block.

4 ZOO DEX

ZOO ships a native AMM DEX. The design is intentionally Uniswap-shaped at the surface so existing tooling, audits, and pool analytics translate directly. Underneath, the curve evaluation runs on the GPU per §3.

4.1 V2 — constant product

V2 implements the canonical $x \cdot y = k$ AMM. Pools are pairs of ZRC-20 tokens; LP shares are minted as ZRC-20 receipt tokens. Fees default to 30 bps with per-pool override. A V2 swap on ZOO 3.0 costs roughly the same gas as a Uniswap V2 swap on a CPU EVM, but block-level throughput is materially higher because hundreds of swaps execute in parallel on the device.

4.2 V3 — concentrated liquidity

V3 implements concentrated liquidity in the Uniswap V3 style [17]: liquidity providers position capital in price ranges defined by tick boundaries. Tick crossings, fee accumulation, and range-position accounting are device-side. Multi-fee-tier pools (5 / 30 / 100 bps) coexist for the same pair.

4.3 V4 — LUX DEX precompile

V4 is not a separate AMM. V4 is a precompile interface that lets ZOO contracts route swaps to the LUX DEX [15], which lives at `~/work/lx/dex`. The LUX DEX is the cross-chain liquidity router and the canonical venue for LUX-ecosystem CLOB-style matching; ZOO V4 plugs into it via B-Chain (LP-134) routed precompile calls. Use V4 when:

- Routing across chains where the deepest book lives on LUX.
- Matching against external order flow that the LUX DEX already aggregates.
- Serving institutional flow that expects CLOB semantics.

Use V2/V3 when:

- Trading ZOO-native assets (per-LLM tokens, wildlife NFT fractional pools, DAO governance pools).
- Latency budget is sub-block; staying chain-local avoids the B-Chain hop.
- LP yield must be programmable per pool by Zoo DAO policy.

4.4 Optional local Zoo DEX precompile

For ZOO-internal use cases that want CLOB semantics without the B-Chain hop, ZOO ships an optional in-chain DEX precompile that implements the same matching primitives as the LUX DEX but against ZOO-local order books. The precompile shares its kernel implementation with LUX DEX so behaviour is identical; only the venue and gas accounting differ.

4.5 Per-LLM token markets

The per-LLM chain framework [12] mints a distinct gas/utility token per ZEN model. These tokens trade against \$ZOO and against each other on ZOO DEX V2/V3. Inference revenue flowing through the per-LLM chain attribution graph settles into the relevant V3 pool, providing a price discovery mechanism for model quality.

4.6 NFT fractional pools

NFTs migrated to ZOO L1 (§9) can be wrapped into ERC-4626 vault shares and traded as concentrated-liquidity V3 positions. Combined with F-Chain FHE (§8), the fractional positions can trade with confidential balances — material for high-value collections.

4.7 Versioning policy

ZOO DEX V2 and V3 are stable. V4 is the precompile interface and will track the LUX DEX interface contract; breaking changes to LUX DEX require a corresponding ZOO DEX V4 revision. There is no separate “Zoo V4 AMM”; V4 is the precompile, full stop.

5 Zoo AI Desktop and Local Private AI

ZOO 3.0 ships a desktop application — the ZOO AI Desktop — that runs the ZEN model family locally on the user’s machine. The application is the user-facing entry point to Zoo’s privacy-first AI posture: weights live on the device; queries do not leave the device unless the user opts in.

5.1 Form factor

The ZOO AI Desktop is a Tauri 2 application (Rust backend, React frontend) that auto-spawns the `zoo-node` runtime on launch. The local node hosts:

- Inference runtimes: `llama.cpp`, `vLLM`, and `ollama` backends, selected by hardware capability.
- Model registry: the local copy of the ZEN family (`zen4-nano` through `zen4-ultra-max`), gated by device VRAM.
- Wallet integration: \$AI mining rewards accrue to the device’s local wallet.
- MCP server: agents in the desktop expose HANZO MCP tools for local-first task automation.

5.2 Free local inference

A user with a recent consumer GPU can run `zen4-nano`, `zen4-mini`, or `zen4-small` fully locally with no network egress. Inference is free at the marginal cost of electricity. The user’s queries do not enter any logging surface, do not enter any cloud telemetry pipeline, and do not enter any third-party datastore.

5.3 Zoo Cloud upgrade path

Users running larger models (`zen4-large`, `zen4-ultra`, `zen4-ultra-max`) that exceed local VRAM can opt into Zoo Cloud, which dispatches to the federated capacity layer documented in [12] (§Pooled GPU Hosting). Zoo Cloud is opt-in, pay-per-token, and TEE-attested: the cloud worker runs in an H100 / TDX / SEV-SNP container, the inference receipt is anchored to A-Chain, and the user can verify post-hoc that the response came from the model the user paid for.

5.4 Privacy posture

The default privacy posture is local-only:

- No cloud calls without explicit user opt-in.
- No telemetry without explicit user opt-in.
- Wallet signing happens locally; the signing key never leaves the device.
- Conversation history is stored encrypted in a local SQLite database (per Zoo Web5 / local-first architecture).

When the user opts into Zoo Cloud, the inference request is routed via LUX F-Chain FHE (§8) when confidential mode is selected, so the cloud worker never sees the plaintext query.

5.5 Mining UX

The desktop’s wallet pane shows live \$AI accrual from compute the device contributes to the Hanzo network. A typical desktop with a recent gaming GPU can earn approximately one \$AI per minute under default mining settings; rates depend on market demand for the device’s compute class.

5.6 Why this matters for Zoo 3.0

The ZOO 3.0 narrative is GPU-native everywhere. The desktop app is the user-side endpoint of that narrative: the same GPU architecture that runs validator-side consensus runs user-side inference. ZOO is the chain whose user runtime and validator runtime share a substrate.

6 zoo-bot

`zoo-bot` is ZOO’s pure-Go re-write of the HANZO OpenClaw bot (at `~/work/hanzo/bot`). The original is a TypeScript agentic bot framework that powers content automation, on-chain triggers, and Discord/Slack integrations across the Hanzo ecosystem. ZOO’s re-write keeps the surface — skills, tools, MCP integration, web hooks — but re-implements the runtime in Go for size, speed, and attack surface.

6.1 Why a re-write

The OpenClaw original is good. It is also a Node runtime carrying its TypeScript ecosystem of dependencies: hundreds of npm packages, an asynchronous event loop, a JIT, and a garbage collector tuned for front-end-style workloads. For agentic workflows that run in continuous-loop mode on a server, this is overhead. `zoo-bot` trades the front-end ergonomic toolchain for a single static binary.

6.2 Goals

1. **Smaller binary.** Single static Go binary. Target <50 MB stripped, versus the original’s roughly 200 MB Node runtime + dependencies.
2. **Lower memory.** Idle footprint under 80 MB resident, versus the original’s typical 250+ MB.
3. **Faster cold start.** Sub-100 ms startup to first accepting connection, versus several seconds for the original.
4. **Smaller attack surface.** No npm dependency tree. No post-install scripts. No transitive supply-chain exposure beyond Go’s standard library plus a vetted set of direct module dependencies.
5. **Drop-in surface.** Agents written for OpenClaw run on `zoo-bot` via a compatibility shim that translates the TS skill manifest to Go’s plugin descriptor.

6.3 Architecture

`zoo-bot` is organised into three layers:

- **Core.** Event loop, scheduler, MCP server/client, tool dispatcher, persistence (SQLite or Postgres).
- **Skills.** Pluggable Go modules implementing the `Skill` interface. Each skill is a small struct with `Plan`, `Execute`, and `Observe` methods.
- **Adapters.** Connectors to Discord, Slack, Matrix, webhook endpoints, and on-chain trigger streams from ZOO L1 / LUX chains.

6.4 Use cases

- **Agentic workflows.** An agent that monitors a per-LLM chain attribution graph and posts contributor leaderboards to Discord.
- **Content automation.** An agent that watches an A-Chain attestation feed and publishes provenance-verified inference outputs to a public mirror.
- **On-chain triggers.** An agent that fires when a wildlife NFT crosses a price threshold or when a per-LLM chain emits a new training-data attestation.
- **Validator-side automation.** An agent that watches the ZOO validator’s mempool, surfaces anomalies, and pages the operator.

6.5 Security model

Each skill runs in a Goroutine sandboxed by a capability set: filesystem paths it can read/write, network endpoints it can dial, RPC endpoints it can call. The sandbox is enforced by a syscall-level hook (Linux seccomp profile, macOS sandbox profile) that the bot loads at startup. Skills cannot escape their declared capabilities without a code modification and a re-deploy.

6.6 Relationship to OpenClaw

OpenClaw remains the upstream reference. `zoo-bot` is a re-implementation that interoperates with the OpenClaw skill manifest format, so a skill author can target both. HANZO continues to maintain OpenClaw; ZOO maintains `zoo-bot`. The two share the MCP wire protocol so a `zoo-bot` agent can call OpenClaw tools and vice versa.

7 Zoo Bridge

Zoo Bridge (at `~/work/zoo/bridge`) is the cross-ecosystem bridge for Zoo L1. It lets ZRC-20 fungibles, ZRC-721 NFTs, ZRC-1155 multi-tokens, and the native \$ZOO gas token move between Zoo L1 and external chains while preserving conservation metadata, provenance chains, and token-bound account state.

7.1 Architecture

Zoo Bridge is layered:

- **Bridge VM layer.** Zoo Bridge sits on top of the LUX B-Chain (BVM, LP-134). The B-Chain is the LUX ecosystem's bridge substrate; Zoo Bridge is the ZOO-specific message and asset model on top of it.
- **Threshold signing.** Lock and mint operations require a 2-of-3 supermajority across the ZOO bridge committee, which signs via M-Chain (MVM) MPC ceremonies (CGGMP21 / FROST), with RINGTAIL threshold proofs anchoring the post-quantum side.
- **Asset adapters.** One adapter per supported foreign chain: LUX C/X/B/Z-Chains, Ethereum mainnet, Polygon, BSC (legacy migration path for the original 2021 ZOO deployment), and arbitrary EVM-compatible chains via the generic adapter.

7.2 Supported chains

At activation, Zoo Bridge supports:

Chain	Direction	Notes
LUX C-Chain	both	native EVM, fastest
LUX X-Chain	both	for UTXO-based assets
LUX B-Chain	both	native bridge VM
LUX Z-Chain	both	for ZK-rollup-bound assets
Ethereum mainnet	both	wrapped \$ZOO, NFT migration
BSC	inbound	legacy 2021 ZOO migration only
Polygon	both	retail wallets and tooling
Arbitrary EVM	both	generic adapter, slower path

7.3 Conservation metadata preservation

A wildlife ZRC-721 carries fields that generic bridges lose: provenance chain (pedigree), conservation contributions, token-bound account balances, and on-chain achievements (gym, breeding, feeding). Zoo Bridge encodes these into the lock event and replays them on the mint side. A bridged NFT arrives identical, not a synthetic wrapper.

7.4 Yield-bearing bridge bookkeeping

When \$ZOO sits on the foreign side as a wrapped token, the locked backing on ZOO L1 can stake into the validator set. Yield is accrued on a per-deposit basis; on burn-and-unlock, the depositor receives principal plus accumulated yield. This follows the yield-bearing bridge pattern (LP-018).

7.5 Security model

The threshold signing ceremony has three properties:

1. **Liveness.** Any 2-of-3 of the bridge committee can sign; one offline signer does not stall the bridge.
2. **Safety.** Any forged release requires compromising 2-of-3 committee members *and* forging a RINGTAIL threshold proof on M-Chain. The RINGTAIL layer is post-quantum.
3. **Auditability.** Every signed message is logged on B-Chain with the full triple-consensus certificate.

7.6 Migration of 2021 Zoo BSC deployment

The original 2021 ZOO deployment lives on BSC. ZOO Bridge provides a one-way migration adapter that reads the BSC ZRC-20 and ZRC-721 holdings, escrows them under the BSC bridge contract, and mints native equivalents on ZOO L1. Users opt in by signing a migration intent with the BSC private key; the bridge committee counter-signs and the L1 token is delivered. The BSC token contract is locked post-migration so legacy holders cannot double-spend.

8 Privacy

ZOO 3.0 supports two complementary privacy paths via LUX chain topology (LP-134): FHE via F-Chain (LP-013) for confidential compute on encrypted data, and ZKP via Z-Chain (LP-063) for selective disclosure of private facts. They are not alternatives; they solve different problems.

8.1 F-Chain FHE (LP-013)

F-Chain (FVM) is a fully-homomorphic encryption execution environment. Contracts on F-Chain operate on ciphertexts: the chain never sees the plaintext, and yet the result of a computation is verifiable on-chain.

Confidential ERC-20 (LP-067). The Confidential ERC-20 standard runs balances and allowances as ciphertexts. A confidential transfer ciphertext-decrypts the source balance, ciphertext-subtracts the amount, ciphertext-adds to the destination, and ciphertext-encrypts the new balances. Validators verify the operation without ever seeing a plaintext balance. Useful for:

- Confidential trading of high-value collectibles where price movements should not be public until settlement.
- Per-LLM token holdings that the holder wants to keep private for governance reasons.
- Treasury balances that the DAO holds confidentially until formal disclosure.

Private inference. A user submits an encrypted prompt to F-Chain; an FHE-aware Zen inference worker performs the forward pass on the ciphertext; the encrypted output is returned to the user, who decrypts locally. The worker never sees the plaintext prompt or the plaintext output. This is the canonical pattern for confidential AI in regulated workflows (healthcare, securities research, internal counsel).

When to use F-Chain. FHE is the right primitive when the property to protect is the *data*, the computation is non-trivial, and verifiers need correctness without seeing the data. FHE costs more than ZKP per operation but supports arbitrary functions on the encrypted data.

8.2 Z-Chain ZKP (LP-063)

Z-Chain (ZVM) is a zero-knowledge proof execution and registry environment. Z-Chain runs a Groth16 / Halo2 / Plonky2 verifier as a GPU kernel and hosts a circuit registry. Contracts on ZOO L1 dispatch verification jobs to Z-Chain via precompile.

Selective disclosure. A user proves “I am over 18” or “I hold $\geq 1,000$ \$ZOO” without revealing date of birth or actual balance. Identity assertions live on A-Chain (AIVM, LP-134); proofs about those assertions live on Z-Chain.

Rollup of identity assertions. For high-volume assertion flows — e.g., per-block KYC checks for a regulated trading venue — Z-Chain rolls up batches of proofs into a single Groth16 verification, amortising verification cost.

When to use Z-Chain. ZKP is the right primitive when the property to protect is a *predicate over data*, the predicate is bounded (compileable to a circuit), and verifiers need a yes/no answer with cryptographic backing. ZKP verification is much cheaper than FHE computation per operation but requires you to know the predicate in advance.

8.3 Choosing between them

Question	F-Chain (FHE)	Z-Chain (ZKP)
Hide values?	yes (whole computation)	no (only the witness)
Hide identity?	via key management	yes (predicate over identity)
Arbitrary compute?	yes	no (must be a circuit)
Per-op cost	high	low to moderate
Setup cost	key generation	circuit compile + trusted setup (Groth16)

The ZOO 3.0 wallet (§10) exposes a privacy mode toggle that picks the right chain for the user’s intent: “hide this transfer” routes to F-Chain Confidential ERC-20; “prove eligibility” routes to Z-Chain selective disclosure.

8.4 GPU-native privacy

Both privacy paths are GPU-native end-to-end. F-Chain’s TFHE bootstrapping runs on the GPU per LP-013. Z-Chain’s Groth16 verification (and the prover for circuits compiled in advance) runs on the GPU. The privacy primitives in ZOO 3.0 are not bolted on at the application layer; they execute on the same substrate as the rest of the chain.

9 NFT Strategy

ZOO 3.0 launches with an explicit go-to-market for NFT migration. ZOO intends to make competitive offers to the leading NFT collections in the wildlife/conservation, AI/agent, and metaverse categories for migration to ZOO L1. The goal is for ZOO to host a top-2 NFT project portfolio inside its first activation cycle.

9.1 Why a collection should migrate

Native FHE privacy. High-value collectibles benefit from confidential trading: the floor price reveals less to predators, holders can build positions without front-running, and treasury balances stay private until disclosure. F-Chain (LP-013) makes this a chain-level primitive, not a per-collection bolt-on.

GPU-fast minting and transfer. Drop mechanics that bottleneck on signature verification clear faster on a GPU-native chain. A 10k-supply drop that exceeds CPU EVM block budgets clears inside a normal block on ZOO 3.0.

Native AI provenance. A-Chain (AIVM, LP-134) anchors TEE-attested inference receipts. For collections whose value derives from generative outputs (AI-art collections, agent NFTs, on-chain creatures with AI behaviour), provenance is a chain-level guarantee, not a marketing claim.

Per-NFT token-bound accounts. ZRC-721 NFTs on ZOO L1 have native ERC-6551-style token-bound accounts. The account can hold \$ZOO, per-LLM tokens, other NFTs, and accrue yield from staking. Combined with the ZOO DEX (§4) and bridge (§7), the NFT becomes a programmable agent rather than a static JPEG.

DAO-aligned governance. Migrated collections plug into Zoo DAO voting weight. Holders participate in conservation grants, per-LLM chain governance, and treasury allocation.

9.2 Migration mechanics

1. **Snapshot.** ZOO Bridge (§7) takes a snapshot of holders on the source chain.
2. **Lock-and-mint.** The source-chain contract is escrowed; the ZOO L1 native ZRC-721 is minted to the same wallet address.
3. **Metadata mirror.** On-chain metadata moves over with provenance hash chain intact; off-chain assets re-pin to ZOO’s decentralised storage layer.
4. **Treasury onboarding.** The collection’s treasury is onboarded to a ZOO DAO-governed multisig with optional F-Chain FHE balance privacy.

5. **Royalty redirection.** Future royalties accrue to the L1 contract; the bridge keeps the source-chain contract readable for legacy explorers.

9.3 What ZOO offers a migrating collection

The migration package includes:

- Validator-tier infrastructure for the collection’s contracts.
- Native integration with the ZOO DEX for floor and fractional liquidity.
- Marketing co-promotion across the ZOO foundation channels.
- Treasury-grant matching for conservation- or AI-aligned collections from the Zoo Fund.
- Optional F-Chain confidential balances and Z-Chain selective disclosure for holder identity.
- Free ZOO AI Desktop integration for collections that ship agent companions.

9.4 Categories

ZOO prioritises three collection categories:

1. **Wildlife and conservation.** Aligned with the Zoo Labs Foundation 501(c)(3) mission. Collections whose ongoing impact accrues to species preservation.
2. **AI agents and generative art.** Collections that benefit from A-Chain provenance and Zen model integration.
3. **Metaverse and gaming.** Collections that map to the Zoo metaverse companion app and gym progression.

9.5 Note on naming

This paper does not name specific target collections; competitive offers are confidential until accepted. The architecture above is designed to be a credible offer for any collection in the listed categories, on terms no other chain can match.

10 LUX Wallet and Desktop

ZOO 3.0 plugs into the LUX Wallet and the LUX Desktop app. The LUX Wallet is the canonical signing surface for LUX-ecosystem chains; the LUX Desktop app is the canonical management UI. ZOO L1 appears as a chain target on both, alongside LUX P/C/X/Q/Z/A/B/M/F- Chains.

10.1 Account / portfolio view

The portfolio screen aggregates balances across LUX ecosystem assets and ZOO-specific assets:

- \$LUX (P-Chain stake, C-Chain balance, X-Chain UTXO).
- \$AI (A-Chain mining rewards, Hanzo network earnings).

- \$ZOO (Zoo L1 native, plus bridged wrappers).
- Per-LLM tokens (one per active per-LLM chain [12]: ZEN4-NANO, ZEN4-MINI, . . . , ZEN4-ULTRA-MAX).
- ZRC-721 NFTs with thumbnails, provenance chain, and token-bound account holdings.
- Optional confidential ERC-20 balances (F-Chain), shown as “encrypted” until the user unwraps locally.

10.2 Send / receive

The send flow signs with the user’s local key, submits to the chosen chain, and surfaces QUASARCert finality progress: classical fast-path acknowledgement first, then RINGTAIL threshold confirmation, then ML-DSA identity proof. The wallet shows the triple-consensus state visually, so the user understands the post-quantum side of the certificate is in place before treating the transfer as final.

10.3 DEX swap

The swap screen exposes a chain selector — LUX C-Chain or ZOO L1 — because the same trading pair can have liquidity on both. The selector remembers per-pair preferences. Behind the scenes:

- C-Chain swap routes via the LUX DEX (LP-009 + [15]).
- Zoo L1 swap routes via ZOO DEX V2 / V3 (§4).
- Cross-chain swap (Zoo asset $\rightarrow$ Lux asset) routes via Zoo DEX V4 (the precompile to LUX DEX) plus a B-Chain bridge hop.

10.4 Staking

The staking screen splits between LUX validator delegation (P-Chain) and ZOO validator delegation (Zoo L1 native staking contract). Slashing risk is shown per-validator with on-chain attestation history pulled from A-Chain.

10.5 Bridge

The bridge screen is a wrapper around the B-Chain bridge VM and ZOO Bridge. The user picks source chain, destination chain, and asset; the wallet computes the bridge route, surfaces the threshold ceremony progress, and shows the final mint event.

10.6 AI inference

The AI inference screen integrates ZOO AI Desktop:

- Local-only mode: queries to a local ZEN model. No network.
- Zoo Cloud mode: queries dispatched to federated GPU capacity, paid in \$AI, attested via A-Chain.
- Hanzo Marketplace mode: queries dispatched to the Hanzo AI Chain inference marketplace [11], paid in \$AI, with HMM (Hanzo Model Marketplace) routing.
- Confidential mode toggle: routes via F-Chain FHE (§8) so neither the worker nor the chain sees the plaintext prompt.

10.7 Privacy mode

A toggle at the top of the wallet enables privacy mode for the active session:

- Transfers route via F-Chain Confidential ERC-20 (LP-067).
- Identity proofs route via Z-Chain selective disclosure (LP-063).
- Wallet displays show ciphertext indicators rather than cleartext balances when in privacy mode.

10.8 Mobile and desktop parity

The LUX Wallet ships in three form factors — browser extension, mobile (iOS/Android), and LUX Desktop — with the same screen inventory. The desktop variant additionally surfaces validator operator panes (block production stats, slashing exposure, threshold-ceremony participation) for users running Zoo or Lux validators.

10.9 Screen inventory summary

The full screen specifications — ASCII mocks, flow diagrams, and edge cases — live in the wallet repo at `~/work/lux/wallet/SCREENS.md`, which is the canonical UX reference for engineering.

11 Activation

ZOO 3.0 activates on December 25, 2025, alongside LUX QUASAR 3.0 (LP-020) and the per-LLM chain framework [12].

11.1 Phase 0 — spec freeze (2025-12-15)

This paper, the ZOO DEX ZIP, the ZOO L1 graduation ZIP, the ZOO AI Desktop ZIP, the `zoo-bot` ZIP, and the ZOO Bridge ZIP all freeze on 2025-12-15. After this date, only spec-clarification PRs are accepted; behaviour-changing PRs slip to 3.1.

11.2 Phase 1 — testnet bring-up (2025-12-15 through 2025-12-22)

- Zoo L1 testnet validators come online with QUASAR 3.0 binaries.
- ZOO DEX V2 and V3 deploy on testnet; V4 precompile target dummied to a mock LUX DEX for parallel testing.
- ZOO Bridge committee runs threshold ceremonies on M-Chain testnet and validates BSC migration adapter end-to-end.
- ZOO AI Desktop release candidate ships to internal testers; Zoo Cloud routing exercised via A-Chain testnet.
- `zoo-bot` alpha ships to ecosystem partners.

11.3 Phase 2 — canary (2025-12-22 through 2025-12-24)

- Zoo L1 canary mainnet brought up with a subset of validators.
- Bridge enabled inbound only (foreign → Zoo L1) for legacy 2021 BSC migrants.
- ZOO DEX pools seeded with foundation liquidity for major pairs.
- LUX Wallet 3.0 release candidate accepts ZOO L1 as a chain target.

11.4 Phase 3 — activation (2025-12-25)

- Zoo L1 mainnet open to all validators.
- ZOO DEX V2 / V3 / V4 production-active.
- ZOO Bridge bidirectional on all listed chains.
- ZOO AI Desktop GA.
- zoo-bot 1.0 GA.
- LUX Wallet 3.0 GA with ZOO L1 enabled by default.
- NFT migration program opens; first cohort of partner collections announced.

11.5 Phase 4 — post-activation roll-forward

ZOO 3.x continues to ship features incrementally without breaking the spec freeze in this paper. Forward-looking work continues against the published QUASARSTM 4.0 (LP-135) parallel-execution spec, which Zoo L1 picks up without a chain-level fork at the appropriate epoch.

11.6 Cross-paper relationships

This paper is the activation document for ZOO 3.0. It cross-references:

- Per-LLM chains [12] — the upstream training, research, and pooled-GPU layer.
- Hanzo AI Chain [11] — the inference marketplace and HMM that ZOO dispatches into.
- Zoo 2025 securities and DAO [13] — the digital-securities positioning and holographic-consensus DAO mechanics.

11.7 One-line summary

ZOO 3.0 is what happens when you take the fastest blockchain ever shipped, put the entire hot path on the GPU, and graduate the result to a sovereign L1 that ships with native private AI, native AMM, native bridge, native confidential compute, and a wallet that exposes all of it. December 25, 2025.

References

- [1] Lux Foundation. LP-009: GPU-Native EVM. Lux Improvement Proposal, December 2025. <https://github.com/luxfi/lps/lp-009>.
- [2] Lux Foundation. LP-013: F-Chain FHE-Protected Inference. Lux Improvement Proposal, December 2025.
- [3] Lux Foundation. LP-020: QUASAR Consensus 3.0 (Triple Consensus: BLS + RINGTAIL + ML-DSA). Lux Improvement Proposal, December 2025.
- [4] Lux Foundation. LP-063: Z-Chain Zero-Knowledge Rollup and ZKP Registry. Lux Improvement Proposal, December 2025.
- [5] Lux Foundation. LP-067: Confidential ERC-20 on F-Chain. Lux Improvement Proposal, December 2025.
- [6] Lux Foundation. LP-105: QUASAR Consensus Family Specification (Photon, Wave, Focus, Nova, Nebula). Lux Improvement Proposal, December 2025.
- [7] Lux Foundation. LP-132: QUASARGPU Execution Adapter for Deterministic GPU Scheduling. Lux Improvement Proposal, December 2025.
- [8] Lux Foundation. LP-133: QUASAR-Native App Stack for Sovereign Appchains. Lux Improvement Proposal, December 2025.
- [9] Lux Foundation. LP-134: LUX Chain Topology (P/C/X/Q/Z/A/B/M/F-Chains). Lux Improvement Proposal, December 2025.
- [10] Lux Foundation. LP-137: GPU-Residency Invariant. Lux Improvement Proposal, December 2025.
- [11] Hanzo AI. The HANZO AI Chain: Inference Marketplace and Commerce Layer. Hanzo Whitepaper, December 2025.
- [12] Zoo Labs Foundation. Per-LLM Chains: One Sovereign Chain per Model on QUASAR 3.0. Zoo Whitepaper, December 15, 2025.
- [13] Zoo Labs Foundation. Zoo 2025: Digital Securities, Quantum-Secure Settlement, and the Holographic Consensus DAO. Zoo Whitepaper, December 15, 2025.
- [14] Zoo Labs Foundation. ZOO DEX: AMM V2/V3 Native, V4 Precompile. Zoo Whitepaper, December 2025.
- [15] Lux Network. LUX DEX Specification. <https://github.com/luxfi/dex>, December 2025.
- [16] Zen LM. The ZEN Model Family: Open-Weight Frontier Models. <https://huggingface.co/zenlm>, December 2025.
- [17] Adams, H., et al. Uniswap v3 Core. Uniswap Whitepaper, March 2021.
- [18] Costan, V., Devadas, S. Intel SGX Explained. IACR Cryptology ePrint Archive, 2016.